

ARRIVO - DOSSIER TECNICO COMPLETO DEFINITIVO

Documento per sviluppatore esterno - contiene tutto il contesto tecnico noto, senza password reali

1. Scopo del documento

Questo PDF permette a uno sviluppatore di prendere in mano Arrivo da zero, senza dover chiedere al proprietario dove si trova qualcosa, come e stato configurato il server, quali strumenti sono stati usati, cosa e reale e cosa e ancora demo.

2. Identita progetto

Voce	Dettaglio
Nome	Arrivo
Tipo	SaaS food-tech multi-tenant
Obiettivo	Discovery ristoranti, prenotazioni, ordini, gestione ristoranti, pannello admin
UI	Italiano
Design	Premium nero/arancione
Priorita	Mobile-first: circa 95% utenti previsti da smartphone

3. Regole comunicazione con proprietario

- Spiegare in modo semplice, come a un bambino di 10 anni.
- Una istruzione alla volta.
- Specificare sempre dove digitare: VPS/server o PC locale.
- Quando si cita uno strumento, spiegare tra parentesi il ruolo semplice.
- Evitare muri di testo.
- Non dare per scontate competenze tecniche.

4. Glossario operativo

Termine	Significato
GitHub	Magazzino online dei file del progetto
VPS/server	Computer online Aruba che mostra il sito
PC Windows locale	Computer personale usato come client/terminale SSH, non come server produzione
Codex	AI consigliata per backend, deploy, Git, build, TypeScript
Claude Code	AI consigliata per UI, UX, responsive, mobile, design
PM2	Tiene acceso il processo Next.js
Nginx	Reverse proxy davanti all'app
PostgreSQL	Database reale utenti/ristoranti
Prisma	ORM tra app e PostgreSQL
NextAuth	Auth prevista per sessioni, credentials e Google OAuth
Nodemailer	Email via Aruba SMTP

5. Infrastruttura completa

Elemento	Valore
Provider VPS	Aruba Cloud VPS
OS VPS	Ubuntu 22.04
Dominio	https://arrivoapp.it
IP VPS attuale	209.227.239.83
Vecchio IP storico rimosso/cache	31.11.35.15
DNS	root domain e www devono puntare al VPS Aruba
HTTPS	Attivo con Certbot
Web server	Nginx
Process manager	PM2
Processo PM2	arrivo
Node.js	20.20.2 rilevato in PM2

PM2 command	npm run start -- -H 0.0.0.0
Repo GitHub	https://github.com/davidetiano1985/arrivo
Branch principale	main
Percorso repo VPS	/root/arrivo
Percorso frontend VPS	/root/arrivo/frontend
SSH user noto	root
SSH host	209.227.239.83 oppure dominio se configurato
Porta SSH	22 se non e stata cambiata

6. Stato reale attuale

Online e funzionante

- Homepage demo online.
- /admin demo online.
- /admin/users demo online.
- Layout admin minimale.
- PM2 attivo.
- Nginx attivo.
- HTTPS attivo.
- GitHub e VPS allineati.

Preparato per app reale

- PostgreSQL installato sulla VPS.
- Database arrivo_db creato.
- Utente database arrivo_user creato.
- Prisma schema creato.
- prisma.config.js creato per Prisma 7.
- .env.local.example creato.
- Pacchetti installati: prisma, @prisma/client, next-auth, @auth/prisma-adapter, nodemailer, bcryptjs.

Non ancora reale

- Migrate Prisma non completato.
- .env.local reale VPS da creare/configurare.
- Seed super admin non eseguito.
- Login reale non implementato.
- Registrazione utenti/ristoranti non implementata.
- Email verifica non implementata.
- Sessioni reali non attive.
- Route admin non protette realmente.
- Frontend online ancora principalmente demo UI.
- Responsive mobile ancora da rifinire.

7. Struttura repository

```

Arrivo/
  frontend/                                # Frontend Next.js attivo in produzione
  src/
    app/
      page.tsx                             # Homepage demo
      admin/
        layout.tsx                         # Layout admin
        page.tsx                           # Admin dashboard demo
        users/
          page.tsx                         # Admin users demo
      data/
        restaurants.ts                     # Ristoranti demo
        users.ts                           # Utenti demo
    prisma/
      schema.prisma                        # Schema Prisma
      prisma.config.js                     # Config Prisma 7
      .env.local.example                   # Template env committato
      .env.local                           # Env reale NON committare
      package.json
      package-lock.json

  backend/                                 # FastAPI scaffold, NON attivo ora
  database/                               # Vecchia base/docker, NON usata ora
  frontend_old/                           # Backup vecchio frontend, NON toccare
  PROJECT_STATUS.md                       # Source of truth operativo
  CLAUDE_PROJECT_MEMORY.md                # Memoria tecnica Claude

```

8. Cartelle da non toccare senza motivo

- frontend_old/ - backup vecchio frontend.
- backend/ - FastAPI scaffold non attivo, non usarlo nei task correnti.
- database/ - vecchio schema/docker non usato ora.
- node_modules/ - mai committare.
- .claude/settings.local.json - file locale Claude, mai committare.

- frontend/.env.local - contiene segreti reali, mai committare.

9. Stack tecnico

Area	Tecnologia	Stato
Frontend	Next.js 14 + React + TypeScript	Produzione
CSS	Tailwind CSS	Produzione
Runtime	Node.js 20.20.2	Produzione
Processo	PM2	Produzione
Proxy	Nginx	Produzione
Database	PostgreSQL	Installato, DB creato
ORM	Prisma 7	Preparato, migrate da fare
Auth	NextAuth	Installato, API da creare
Email	Nodemailer + Aruba SMTP	Installato, servizio da creare
Hash password	bcryptjs	Installato
Backend Python	FastAPI scaffold	Presente ma non usato

10. Database PostgreSQL

Campo	Valore
Database	arrivo_db
Utente	arrivo_user
Password	Non nel PDF. Solo in frontend/.env.local sulla VPS
Host	localhost sulla VPS
Porta	5432
Dove eseguire migrate/seed	Solo sulla VPS via SSH

```
Comandi già eseguiti:
sudo apt update
sudo apt install postgresql postgresql-contrib -y
sudo systemctl status postgresql
sudo -u postgres createuser arrivo_user --pwprompt
sudo -u postgres createdb arrivo_db -O arrivo_user
sudo -u postgres psql -l
```

11. Variabili ambiente necessarie

Il file reale frontend/.env.local deve essere creato sulla VPS e non deve mai essere committato.

```
DATABASE_URL="postgresql://arrivo_user:PASSWORD_REALE@localhost:5432/arrivo_db"
NEXTAUTH_SECRET="generare con: openssl rand -base64 32"
NEXTAUTH_URL="https://arrivoapp.it"

GOOGLE_CLIENT_ID=""
GOOGLE_CLIENT_SECRET=""

SMTP_HOST="smtps.aruba.it"
SMTP_PORT="465"
SMTP_SECURE="true"
SMTP_USER="noreply@arrivoapp.it"
SMTP_PASS="password casella Aruba"
SMTP_FROM="Arrivo <noreply@arrivoapp.it>"
```

12. Prisma 7 - note obbligatorie

- Prisma 7 non usa url dentro datasource in schema.prisma.
- schema.prisma deve avere datasource db con provider = postgresql.
- DATABASE_URL viene caricata tramite prisma.config.js.
- prisma.config.js è obbligatorio per migrate.
- .env.local deve esistere prima di migrate.

```
Corretto:
datasource db {
  provider = "postgresql"
}
```

```
Comandi locali possibili:
npx prisma format
npx prisma generate
```

```
Comandi da fare sulla VPS:
npx prisma migrate dev --name init
```

```
npx prisma db seed
```

13. Workflow Git/deploy

Claude/Codex -> git commit + git push -> GitHub -> VPS git pull -> npm run build -> pm2 restart arrivo

Deploy VPS:
cd /root/arrivo
git pull

cd /root/arrivo/frontend
npm run build

pm2 restart arrivo

Controlli:
pm2 list
pm2 describe arrivo
pm2 logs arrivo --lines 100

cd /root/arrivo
git status
git log --oneline -5

14. PM2 e runtime

- Processo PM2: arrivo.
- PM2 esegue npm run start -- -H 0.0.0.0.
- cwd processo: /root/arrivo/frontend.
- Node rilevato: 20.20.2.
- Per verificare usare pm2 describe arrivo.
- Non creare un secondo processo PM2 per FastAPI ora.

15. DNS e SSL

- Dominio arrivoapp.it punta a 209.227.239.83.
- www.arrivoapp.it deve puntare a 209.227.239.83.
- Vecchio IP 31.11.35.15 è stato rimosso ma era rimasto in cache storica.
- HTTPS attivo con Certbot.
- Nginx già configurato.

16. Ruoli e super admin

Ruolo	Significato
user	Cliente normale
restaurant_admin	Gestore ristorante
super_admin	Controllo totale piattaforma

- Super admin previsto: Davide Tiano.
- Username iniziale desiderato: davide.
- Password iniziale desiderata: non salvare in chiaro, hashare con bcrypt.
- Il super admin deve poter cambiare username e password dal pannello.
- Nessuna credenziale deve restare hardcoded nel frontend.

17. Funzioni reali desiderate

Utenti

- Registrazione reale email/password.
- Login reale.
- Google OAuth.
- Verifica email tramite link.
- Nessuna approvazione admin per clienti normali.

Ristoranti

- Registrazione ristorante reale.
- Stato iniziale pending.
- Approvazione/rifiuto da super admin.
- Visibile solo se approved.

Email

- Aruba SMTP.
- Mittente: noreply@arrivoapp.it.
- Uso iniziale: verifica email.
- Uso futuro: reset password e notifiche.

18. UI e responsive

- UI italiana.
- Design nero/arancione premium.

- Mobile-first obbligatorio.
- Niente scroll orizzontale su smartphone.
- Niente tabelle desktop su mobile: usare card verticali.
- Bottoni e select touch-friendly.
- Email, ID e testi lunghi devono andare a capo.
- Responsive mobile non ancora perfetto, da rifinire.

19. AI e autorizzazioni

- Claude Code: UI, UX, responsive, mobile, design.
- Codex: backend, Git, deploy, VPS, build, TypeScript.
- Evitare task che chiedono molti Consenti.
- Non avviare localhost, browser, next dev/start o porte 3000/3001 senza richiesta esplicita.
- Non fare deploy automatico senza conferma.
- Non fare push senza che il task lo dica chiaramente.

20. Problemi risolti

- DNS vecchio IP 31.11.35.15 rimosso/cache risolta.
- Certbot/HTTPS completato.
- 404 /admin causato da file non pushati su GitHub/VPS: risolto.
- VPS package-lock.json sporco: risolto con git restore.
- Prisma 7 config sistemata con prisma.config.js.
- PostgreSQL installato, database e utente creati.

21. Prossimi step precisi

1. Creare frontend/.env.local sulla VPS.
2. Inserire DATABASE_URL reale e NEXTAUTH_SECRET.
3. Eseguire sulla VPS:


```
cd /root/arrivo/frontend
npx prisma generate
npx prisma migrate dev --name init
```
4. Verificare tabelle: User, Restaurant, Account, Session, VerificationToken.
5. Creare seed super admin.
6. Eseguire seed sulla VPS.
7. Creare API NextAuth.
8. Creare login/register.
9. Creare verifica email con Nodemailer/Aruba.
10. Creare registrazione ristorante pending.
11. Creare approvazione ristoranti da admin.
12. Proteggere /admin con sessione e ruolo super_admin.
13. Convertire admin/users da demo a DB reale.

22. Comandi utili

```
# Pulire schermo terminale
clear

# Stato PostgreSQL
sudo systemctl status postgresql

# Lista database
sudo -u postgres psql -l

# Stato Git
cd /root/arrivo
git status

# Pull
git pull

# Build
cd /root/arrivo/frontend
npm run build

# Restart
pm2 restart arrivo

# Dettagli processo
pm2 describe arrivo
```

23. Sicurezza

- Non pubblicare questo PDF online.
- Non contiene password reali.
- Condividere solo con sviluppatore fidato.
- Password e token solo in .env.local sulla VPS.
- Ogni permesso reale deve essere validato lato server.
- Il frontend non deve mai essere unica fonte di autorizzazione.